

INF-08c Podprogramy s parametry a návratovou hodnotou

30 minut
výstup ŠVP INF-08c**INF-08c** V programu používám podprogramy s parametry a návratovými hodnotami.

Na test máš 30 minut. Pracuj samostatně. Programy piš v Pythonu (Google Colab nebo IDLE) a můžeš je spouštět; u úloh „Co program vypíše?“ nejdřív odpověz bez počítače a teprve potom si výstup ověř. Nepoužívej AI asistenta. Odpovědi a hotové programy (soubor nebo odkaz na Colab) nahraj do portfolia na Google Disku.

1. Co program vypíše? Kolikrát se při běhu programu zavolá funkce obsah?

```
def obsah(a, b):  
    return a * b  
  
def cena_podlahy(delka, sirka, cena_m2):  
    return obsah(delka, sirka) * cena_m2  
  
print(obsah(3, 4))  
print(cena_podlahy(4, 2.5, 600))  
print(cena_podlahy(2, 3, 450) + cena_podlahy(1, 1, 450))
```

2. Co program vypíše? Vysvětli, proč skončí chybou, a oprav funkci tak, aby program vypsál 11.

```
def dvojnásobek(x):  
    print(2 * x)  
  
y = dvojnásobek(5)  
print(y + 1)
```

3. Urči parametry funkce a argumenty při jednotlivých voláních. Co program vypíše a proč poslední řádek skončí chybou?

```
def pozdrav(kdo, pocet):  
    zprava = "Ahoj, " + kdo + "! "  
    return zprava * pocet  
  
print(pozdrav("světe", 2))  
print(pozdrav("Pythone", 1))  
print(zprava)
```

4. Napiš funkci vzdálenost(x_1, y_1, x_2, y_2), která vrátí vzdálenost bodů $[x_1; y_1]$ a $[x_2; y_2]$, a funkci obvod_trojuhelniku(ax, ay, bx, by, cx, cy), která ji třikrát použije. Spočítej vzdálenost bodů [0; 0] a [3; 4] a obvod trojúhelníků s vrcholy [0; 0], [3; 0], [3; 4] a [1; 1], [4; 5], [-2; 5].
5. Program „plánovač úspor“ načte cílovou částku a měsíční vklad (kladná čísla), zjistí, za kolik měsíců cíle dosáhneš, a vypíše výsledek. Navrhni rozdělení programu na tři funkce – u každé napiš název, parametry, návratovou hodnotu a co dělá – a hlavní část programu, která je volá. Proč je lepší, když výpočetní funkce nepoužívá input() ani print()?
6. Napiš funkci statistika(s), která pro neprázdný seznam čísel vrátí trojici (minimum, maximum, průměr) bez použití min, max a sum. Co potom vypíše tento kód?

```
print(statistika([4, 8, 6]))  
print(statistika([10, 20]))  
mn, mx, pr = statistika([-3, 7, 2, 2])  
print(mx - mn, pr)
```

7. Co program vypíše? Uprav funkci tak, aby pro prázdný seznam vrátila None, a napiš aspoň tři testy pomocí assert (jeden z nich pro prázdný seznam).

```
def prumer(s):  
    return sum(s) / len(s)  
  
print(prumer([2, 4, 9]))  
print(prumer([]))
```

Test slouží k doložení výstupu do portfolia. Nehodnotí se body ani známkou – rozhoduje, jestli splníš kritéria na konci dokumentu.

Klíč a řešení

- 12 / 6000.0 / 3150; funkce obsah se zavolá 4× (jednou přímo, třikrát uvnitř cena_podlahy)** – ověřuje: volání funkce s různými argumenty a použití funkce uvnitř jiné funkce
 1. obsah(3, 4) = 12.
 2. cena_podlahy(4, 2.5, 600) = obsah(4, 2.5) · 600 = 10.0 · 600 = 6000.0 (desetinné číslo kvůli 2.5).
 3. cena_podlahy(2, 3, 450) = 6 · 450 = 2700, cena_podlahy(1, 1, 450) = 450; součet 3150.
- Vypíše 10 a pak skončí chybou TypeError: funkce hodnotu jen vypíše, nevrací ji – vrací None a None + 1 nejde spočítat. Oprava: v těle funkce return 2 * x místo print(2 * x); pak y = 10 a program vypíše 11.** – ověřuje: rozdíl mezi výpisem (print) a návratovou hodnotou (return)
- Parametry: kdo, pocet; argumenty: "světe", 2 a "Pythone", 1. Výstup: „Ahoj, světe! Ahoj, světe! “ a „Ahoj, Pythone! “, pak chyba NameError – proměnná zprava je lokální, existuje jen uvnitř funkce.** – ověřuje: parametr × argument, lokální proměnná
- 5.0; obvody 12.0 a 16.0** – ověřuje: návrh funkcí s parametry a návratovou hodnotou a jejich opakované použití
 1. return math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2) – vzdálenost podle Pythagorovy věty.
 2. Obvod = vzdálenost(A, B) + vzdálenost(B, C) + vzdálenost(C, A).
 3. První trojúhelník: 3 + 4 + 5 = 12.
 4. Druhý: $|AB| = \sqrt{9+16} = 5$, $|BC| = 6$, $|CA| = \sqrt{9+16} = 5$, obvod 16.
- Např. nacti_kladne_cislo(vyzva) → float (opakuje dotaz, dokud uživatel nezadá kladné číslo; použije se 2×); pocet_mesicu(cil, vklad) → int (cyklem nebo zaokrouhlením podílu nahoru); vypis_vysledek(mesice) → None (vypíše větu). Hlavní část: cil = nacti_kladne_cislo("Cíl (Kč): "), vklad = nacti_kladne_cislo("Vklad (Kč): "), vypis_vysledek(pocet_mesicu(cil, vklad)). Výpočetní funkci bez input a print lze použít s libovolnými daty a snadno otestovat (assert).** – ověřuje: rozdělení programu na podprogramy a návrh jejich rozhraní
- (4, 8, 6.0) / (10, 20, 15.0) / 10 2.0** – ověřuje: funkce vracející více hodnot a její použití s různými vstupy
 1. nejmensi = s[0], nejvetsi = s[-1], soucet = 0; v cyklu přičítat a porovnávat.
 2. return nejmensi, nejvetsi, soucet / len(s) – vrací trojici.
 3. [-3, 7, 2, 2]: minimum -3, maximum 7, průměr $\frac{8}{4} = 2$; rozdíl $7 - (-3) = 10$.
- 5.0 a pak chyba ZeroDivisionError (dělení nulou pro prázdný seznam). Oprava: if len(s) == 0: return None na začátku funkce. Testy např.: assert prumer([2, 4, 9]) == 5; assert prumer([7]) == 7; assert prumer([]) is None.** – ověřuje: ošetření vstupu ve funkci a testování funkce

Kritéria splnění výstupu

- Určí výstup programu s voláním funkcí včetně volání funkce uvnitř jiné funkce (úloha 1).
- Rozliší print a return, parametr a argument a vysvětlí, proč lokální proměnná není mimo funkci vidět (úlohy 2 a 3).
- Napiše funkce s parametry a návratovou hodnotou, které dávají správné výsledky pro všechny zadané vstupy, a jednu z nich použije opakovaně (úlohy 4 a 6).
- Navrhne rozumné rozdělení programu na funkce s popsány parametry a návratovými hodnotami (úloha 5).
- Ošetří neplatný vstup ve funkci a ověří ji testy (úloha 7).

Pro učitele: varianty a přizpůsobení

Obměny: jiné funkce k napsání (prevod_tploty(t, jednotka), je_prvocislo(n), kombinace(n, k) pomocí faktorial(n)), v úloze 5 rozklad jiného programu (kvíz, kalkulačka ceny cesty). Pro silnější žáky: rekurzivní funkce a porovnání s cyklem, výchozí hodnoty parametrů. Pro slabší: v úloze 4 dát hlavičky funkcí a nechat doplnit jen těla. Test lze zadat jako Colab, podkladem do portfolia je odkaz s funkcemi a proběhlými testy.

Výstup se probírá v lekcích: 1.+2. H52 Podprogramy (týden 26)